COMPSCI 701:
Creating Maintainable Software

Lecture 6.1: Comprehensible Classes

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Admin
  - Schedule:
    - Monday — lecture 06.1
    - Tuesday — lecture 06.2
    - Wednesday — review (provide questions on Canvas Discussion)
    - Friday (29th) — Test 1, A02 due
  - Questions?
- Assignment 1 Feedback
- Class Exercise
- Comprehensible classes

Assignment 1 Feedback

- Functionality — Most implementations met functional requirements
- Object-oriented design
  - About 30% not meeting the minimum requirement of 6 classes, of which 3 had to have more than one object created during an execution
- Maintainability
  - Was there care shown to make the code easy maintain for others?
  - 57/97 had one or more PMD violations (more tomorrow)
- Bonuses — 66 early completion, most showed reasonable attempt at refactoring

Previously in COMPSCI701

- An *object-oriented program* is one that **when it executes** creates objects that send messages to each other.

  > "...the most important aspect of OOP is the creation of a universe of largely autonomous interacting agents." Timothy Budd 2001 "An Introduction to Object-Oriented Programming (3rd Ed)" Pearson

- An object-oriented design is one that describes an object-oriented program.

Class Exercise

- Question: Can all designs come from an existing implementation through refactoring?
- Specifically, can the classes in `NoughtsAndCrosses-01.3` be found by refactoring `NoughtsAndCrosses-02.1`?
- Start with `ASCIIDisplay` (if there is time consider others)

Comprehensible Classes

- Classes should “represent” “concepts” from the context schema or design schema
- What is a “concept”?
- How does we know a class “represents” the concept?

Martin on Classes

- Organisation
- Encapsulation
- Small
- Single Responsibility Principle (SRP)
- Cohesion

Organisation

- "Standard" order
  1. public static final fields (constants)
  2. other static fields
  3. instance fields
  4. Constructors
  5. public methods
     - what order?
  6. non public methods (but those called by public methods close to the methods they're called from
     - not always possible
     - what about protected versus private?
- Why is this order the "best" for comprehensibility?

Encapsulation

- fields and “utility functions” usually private, but not fanatical about it — sometimes *protected* is acceptable (Martin, p282)
- “There is seldom a reason to have a public [field]”
- Comments:
  - explaining “good encapsulation” in terms of visibility of fields and methods is inadequate
  - how does making fields and utility methods private improve comprehensibility?

Purpose of Encapsulation

- Objects (and the classes they belong to) represent concepts or abstractions that appear in the context or design schema
- Fields describe the *representation* of the *state* of objects
- Direct access to representation details of abstractions from outside the abstractions is a Bad Idea — thou shalt not expose one's implementation details
  - If the implementer changes the internal representation then everything that directly uses it must change too — a maintenance nightmare
  - But is this comprehensibility or some other aspect of maintainability?
- But, sometimes it "makes sense" to get the values of fields — when is it "good design" to do so?

Encapsulation and Comprehensibility

- Objects represents concepts
- We have expectations as to the behaviour of those concepts, so we expect the objects to behave the same way
- $\Rightarrow$ any deviation of the behaviour of the objects from our expectations creates problems with comprehension
- For example:

```
class Money {
  public int dollars;
  public int cents;
  public Money(int d, int c) { ... }
  public Money add(Money other) { ... }
}
```

We don't expect the values of fields of `Money` to change but by making the fields public, there is the possibility that they can

But

```
class PlayingCard {
  public boolean isFaceUp;
   ...
}
```

we might expect the value of `isFaceUp` to change, so the fact that the field is public is perhaps not so much of a problem

for Comprehensibility at least — there may be problems for other aspects of Mainainability

Getters (and Setters) and Encapsulation

```
public class Money {
  private int _centsOnly; // represent internally with just cents
  public Money(int dollars, int cents) {
    _centsOnly = cents + 100*dollars;
  }
  // "getter" - good or bad?
  public int getCentsOnly() {
    return _centsOnly;
  }
  // Other methods omitted
}
```

- Advice is often given of the form “Proper encapsulation is to make fields private and provide accessor functions (a.k.a getters and settors)”
- But, providing access to fields through getters (and setters) does not guarantee “proper encapsulation”
- A getter that provides information about the choice of representation but otherwise does not support the abstraction the class is supposed to represent “breaks encapsulation”
- A getter that allows objects to behave in a way that is not consistent with our expectation from the concept that object is meant to represent “breaks encapsulation”
- The design question should never be “Should getters and/or setters’ be provided?” It should be What methods are needed to support the abstraction (and if they are getters or setters then so be it)?

Small

- Martin: “The first rule of classes is that the should be small. The second rule is that they should be smaller than that.”
- He does not say what bad thing happens if classes are not small
- We know the less an object (and so the class) represents a concept in the context schema or the design schema, the less comprehensible the design
- One way a class may fail to represent a concept is that it actually represents multiple concepts
- Observation: the smaller the class, the more likely it will match a single concept

Single Responsibility Principle (SRP)

- One of the so-called “SOLID” principles promoted by Martin (and others)
- A class should have only one responsibility (whatever that is. . .)
- A class should have one, and only one, *reason to change*
- Classes that only have one responsibility or only one reason to change are likely to be small

Example


```
1  class Money {
2    private int _cents;
3    private int _dollars;
4    public Money(int dollars, int cents) {
5      _dollars = dollars;
6      _cents = cents;
7    }
8    public String toString() {
9      return "$" + _dollars + "." + _cents;
10   }
11   public String formalString() {
12     return _dollars + "." + _cents + " NZD";
13   }
14 }
```

- “represents money” — one responsibility? one concept?
- Two methods = two responsibilities?
- Two methods, two fields = 4 reasons to change?

Example


```
1  class Money {
2    private int _cents;
3    private int _dollars;
4    public Money(int dollars, int cents) {
5      _dollars = dollars;
6      _cents = cents;
7    }
8    public String toString() {
9      return "$" + _dollars + "." + _cents;
10   }
11   public String formalString() {
12     return _dollars + "." + _cents + " NZD";
13   }
14 }
```

- Why we might want to change this class:
  - Change the way the value of the money is represented (e.g. just the total number of cents) $\Rightarrow$ doesn't change what the class does, what responsibilities it has
  - Change the formats of the strings $\Rightarrow$ while the strings can be changed independently this does not seem to be a significant change to what the class does
  - Change the currency code from NZD to AUD $\Rightarrow$ this changes what the class does
  - Change the symbol from "$" to "€" $\Rightarrow$ this changes what the class does

Cohesion

- **Cohesion:** . . . the extent to which its individual components are needed to perform the same task. . . . how tightly bound or related [a module's] internal elements are to one another *(Yourdon and Constantine, 1979)*
- The less tightly bound the internal elements, the more disparate the parts to the module, the harder it is to understand
- $\Rightarrow$ The higher the cohesion the "better".
- Usually associated with "coupling"
- A class with one responsibility is cohesive
- Classes with many fields and methods that only use a small number of them (different fields for different methods) does not seem very tightly bound

Class “Quality”

- Three views of what it means for a class to be good:
  - Abstraction (matches a concept from the context or design schema)
  - Single Responsibility Principle
  - Cohesion
- Are these all the same?
  - If so, why have all three?
  - If not, what’s the difference (and which should we use)?

Key Points

- To aid comprehension, we want classes that are a good match to concepts in the context and design schema
- Encapsulation means the way an object actually behaves is consistent with the concept it represents
- Various phrases are used to describe objects that are a good match: Abstraction, SRP, Cohesion
- Classes with good objects tend to be “small” (for some definition of small)